

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Структурная и функциональная организация
вычислительных машин

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

на тему

«Организация кэш-памяти»

Выполнил
студент гр. 250541

Власов Р.Е.

Проверил
доцент, к.т.н. каф. ЭВМ

А.А. Воронов

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы	3
2 Исходные данные к работе.....	4
3 Теоретические сведения	5
4 Выполнение работы	7
5 Вывод.....	26

1 Цель работы

Целью лабораторной работы является разработка и проверка кэш-памяти с прямым отображением для индивидуального варианта 4. Разрабатываемое устройство должно принимать запросы процессорного узла, определять попадание или промах, обращаться к основной памяти при отсутствии нужного слова в кэше и возвращать данные с признаком готовности ответа.

В ходе выполнения необходимо определить параметры кэша по таблице вариантов, выбрать разбиение адреса на тег и индекс, описать назначение внешних сигналов, разработать алгоритм обработки чтения и записи, собрать проект в Quartus II 9.1, выполнить функциональную проверку и оценить результаты синтеза.

Таблица 1 – Этапы выполнения лабораторной работы

№	Этап	Результат
1	Анализ индивидуального варианта	Выделены тип отображения, число строк, размер строки и политика замещения.
2	Разработка структуры устройства	Определены основные блоки: разбор адреса, массив строк кэша, сравнение тега, контроллер и основная память.
3	Описание сигналов	Заданы входы процессорного интерфейса, выходные признаки hit/miss/ready и диагностические линии памяти.
4	Разработка алгоритма	Определена последовательность действий при чтении, записи, попадании, промахе и замещении строки.
5	Проверка работы	Сформированы контрольные обращения и получены временные диаграммы.
6	Синтез проекта	Получены показатели использования ресурсов и подтверждено отсутствие ошибок компиляции.

2 Исходные данные к работе

Требуется разработать кэш-память с прямым отображением. В таблице задания для этого варианта указано восемь строк кэша, одна строка в наборе и отсутствие отдельного алгоритма замещения. Это означает, что каждый адрес основной памяти может попасть только в одну строку кэша, а при конфликте новая строка перезаписывает прежнее содержимое выбранного индекса.

Размер строки в задании допускается выбирать из значений 2, 4 или 8 байт. В работе выбран вариант с одной 16-разрядной ячейкой данных, то есть 2 байта на строку. Такой выбор согласуется с общей разрядностью тракта данных микро-ЭВМ и позволяет наглядно проверить чтение, попадание, промах, сквозную запись и конфликт индекса.

Таблица 2 – Исходное задание

Параметр	Значение	Пояснение
Тип кэша	Прямое отображение	Адрес выбирает единственную строку по полю индекса.
Количество строк	8	Для выбора строки используются три младших разряда адреса.
Размер строки	2 байта	В строке хранится одно 16-разрядное слово данных.
Количество строк в наборе	1	Дополнительный выбор пути не выполняется.
Замещение	Не требуется	При совпадении индекса новая строка автоматически заменяет старую.
Политика записи	Сквозная запись	Записываемое слово передается и в кэш, и в основную память.

Внешний интерфейс устройства содержит тактовый вход, вход сброса, запрос процессора, признак записи, адрес, входные данные для записи, выходные данные чтения и признаки состояния. Дополнительно выведены диагностические линии обращения к основной памяти: запрос, запись, адрес

и слово данных. Эти линии используются для подтверждения того, что при попадании основная память не запрашивается, а при промахе формируется цикл чтения или записи.

3 Теоретические сведения

Кэш-память является промежуточным быстродействующим запоминающим устройством между процессором и основной памятью. Ее назначение заключается в хранении копий часто используемых слов, чтобы повторное обращение к тем же адресам выполнялось быстрее. При обращении процессора к адресу контроллер кэша проверяет, находится ли требуемое слово в одной из строк кэша. Если слово найдено, возникает попадание; если нет, фиксируется промах и выполняется обращение к основной памяти.

При прямом отображении каждая строка основной памяти может быть размещена только в одной строке кэша. Номер этой строки определяется индексом, выделенным из младшей части адреса. Такой способ проще аппаратно, чем полностью ассоциативная или множественно-ассоциативная организация, потому что проверяется только один тег и не требуется выбирать одну строку из нескольких возможных путей.

Для восьми строк кэша поле индекса имеет ширину три бита. В данной работе используется 16-разрядный адрес: $A[2..0]$ выбирают строку кэша, а $A[15..3]$ образуют тег. В строке дополнительно хранится valid-бит. Если valid-бит равен единице и сохраненный тег совпадает с тегом текущего адреса, данные считаются корректными и выдаются процессору без обращения к основной памяти.

При промахе контроллер обращается к основной памяти. Полученное слово записывается в строку, выбранную индексом, одновременно обновляются тег и valid-бит. Если в этой строке уже находились данные с другим тегом, они замещаются. Поэтому конфликт адресов 0010h и 0018h удобно использовать как контрольный случай: оба адреса имеют одинаковый индекс, но разные теги.

Для операции записи выбрана политика сквозной записи. При такой политике данные не остаются только в кэше, а сразу отправляются в основную память. Это упрощает согласование кэша и памяти: после записи $W[0010h]=ABCDh$ последующее чтение адреса 0010h после вытеснения должно вернуть ABCDh уже из основной памяти.

Таблица 3 – Разбиение адреса и признаки строки кэша

Поле	Разрядность	Назначение
TAG	13 бит	Старшая часть адреса A[15..3], сохраняется в строке и сравнивается при обращении.
INDEX	3 бита	Младшая часть адреса A[2..0], выбирает одну из восьми строк.
DATA	16 бит	Слово данных, возвращаемое процессору при чтении.
VALID	1 бит	Признак того, что строка содержит корректное слово и тег.
HIT	1 бит	Совпадение valid-бита и тега выбранной строки.
MISS	1 бит	Отсутствие корректной строки для текущего адреса.

Прямое отображение имеет важное преимущество для учебной реализации: путь проверки короткий и легко прослеживается на схеме. Контроллеру не нужно параллельно просматривать несколько строк набора и выбирать подходящий путь. Достаточно по индексу обратиться к одной строке, проверить ее valid-бит и сравнить один сохраненный тег с тегом текущего адреса.

Недостатком прямого отображения является возможность конфликтных промахов. Если две разные ячейки основной памяти имеют одинаковый индекс, они не могут одновременно находиться в кэше. Поэтому обращение к адресу 0018h вытесняет данные адреса 0010h, несмотря на то что остальные строки кэша могут быть свободны. Этот недостаток специально используется в проверке, потому что он демонстрирует основной принцип замещения в кэше прямого отображения.

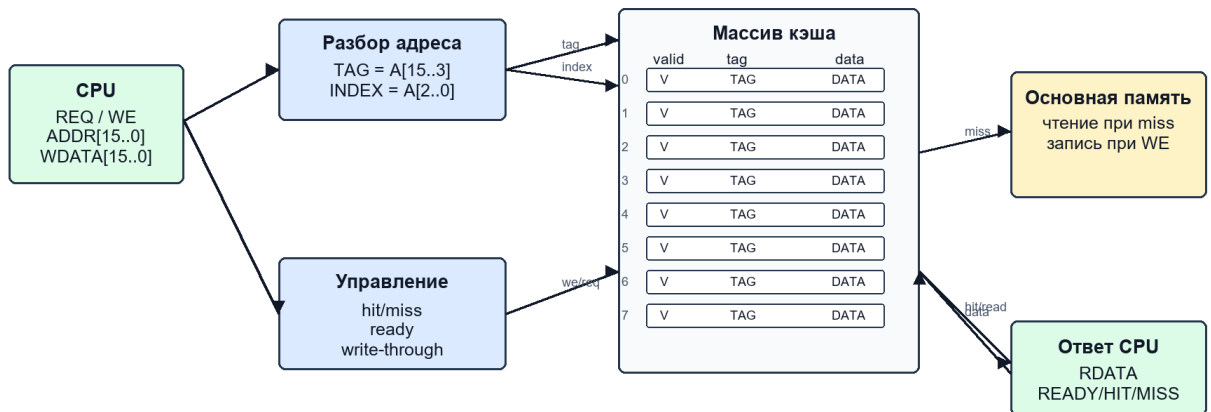
В лабораторной работе важны не только итоговые данные на выходе, но и служебные признаки. Признак hit показывает, что результат получен из строки кэша. Признак miss показывает необходимость обращения к основной памяти. Признак ready отделяет момент принятия запроса от момента готовности ответа. Совместное наблюдение этих сигналов позволяет подтвердить, что устройство работает не как простая память, а как полноценный контроллер кэширования.

4 Выполнение работы

4.1 Структура разрабатываемого устройства

Структура устройства выбрана модульной. Верхний уровень соединяет контроллер кэша и синхронную модель основной памяти. Контроллер принимает запрос процессора, разбирает адрес на тег и индекс, проверяет выбранную строку, формирует признаки попадания и промаха, а также управляет обращением к основной памяти.

Внутри кэша используются массивы valid, tag и data. Массив valid показывает, занята ли строка корректными данными. Массив tag хранит старшую часть адреса. Массив data содержит 16-разрядное слово. Для варианта с прямым отображением отдельный массив возраста или приоритета не требуется, потому что на каждый индекс существует только одна строка.



При совпадении valid и tag формируется hit. При miss выбранная индексом строка перезаписывается данными из основной памяти.

Рисунок 4.1 – Структурная схема кэш-памяти с прямым отображением

На структурной схеме видно, что процессорный интерфейс отделен от интерфейса основной памяти. Такой подход позволяет наблюдать не только итоговые данные чтения, но и внутренний факт обращения к памяти. Если запрос завершается попаданием, линия запроса к памяти остается неактивной. Если возникает промах, контроллер выставляет адрес памяти и ожидает готовность данных.

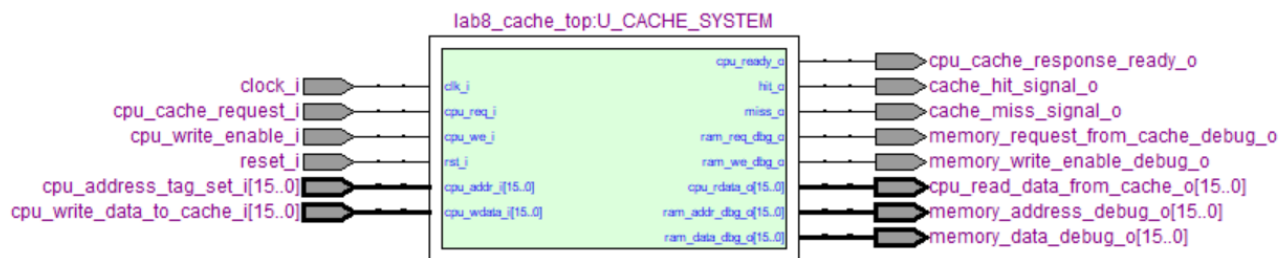


Рисунок 4.2 – Схема верхнего уровня устройства и внешних сигналов

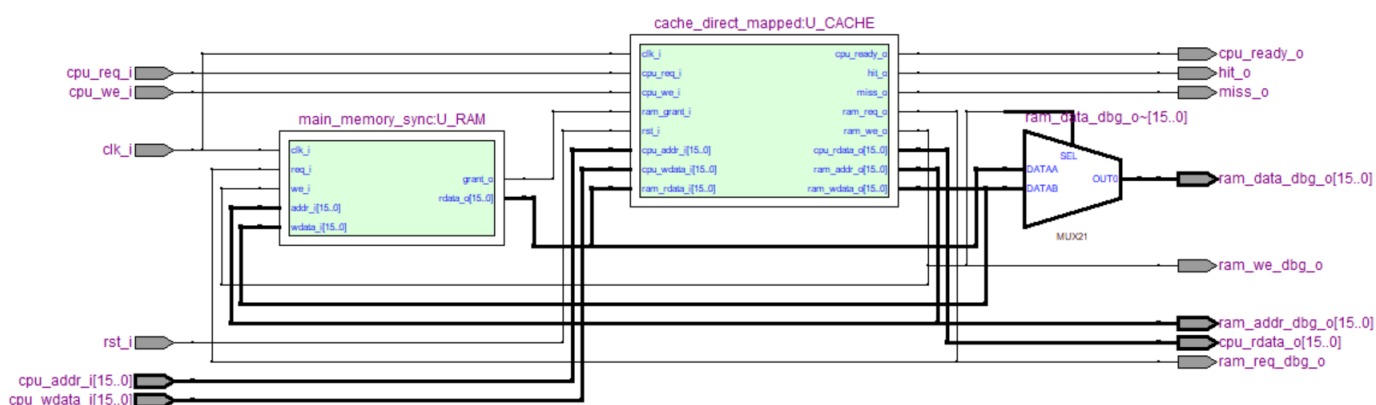


Рисунок 4.3 – Соединение контроллера кэша и основной памяти

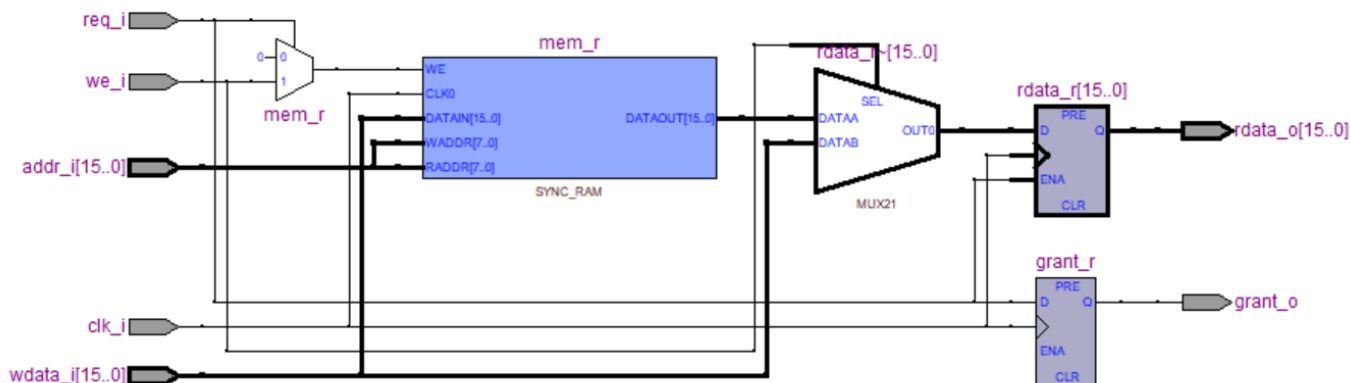


Рисунок 4.4 – Схема синхронной основной памяти

Схемы показывают, что кэш и основная память работают от общего тактового сигнала. Основная память возвращает данные синхронно, поэтому при промахе чтения ответ процессору формируется не в тот же момент, когда поступил запрос, а после завершения цикла обращения к памяти.

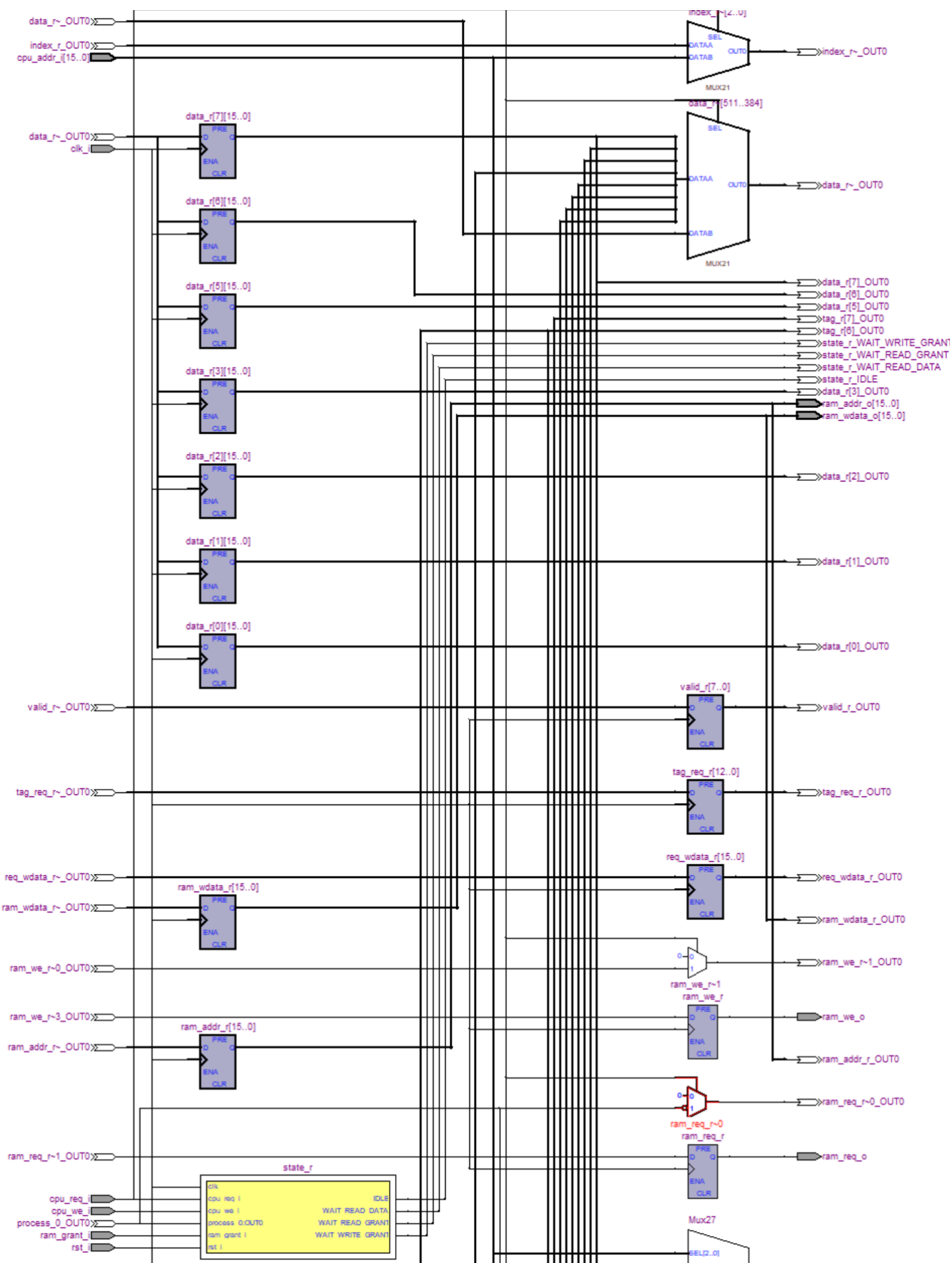


Рисунок 4.5 – Внутренняя организация строк и регистров кэш памяти, часть 1

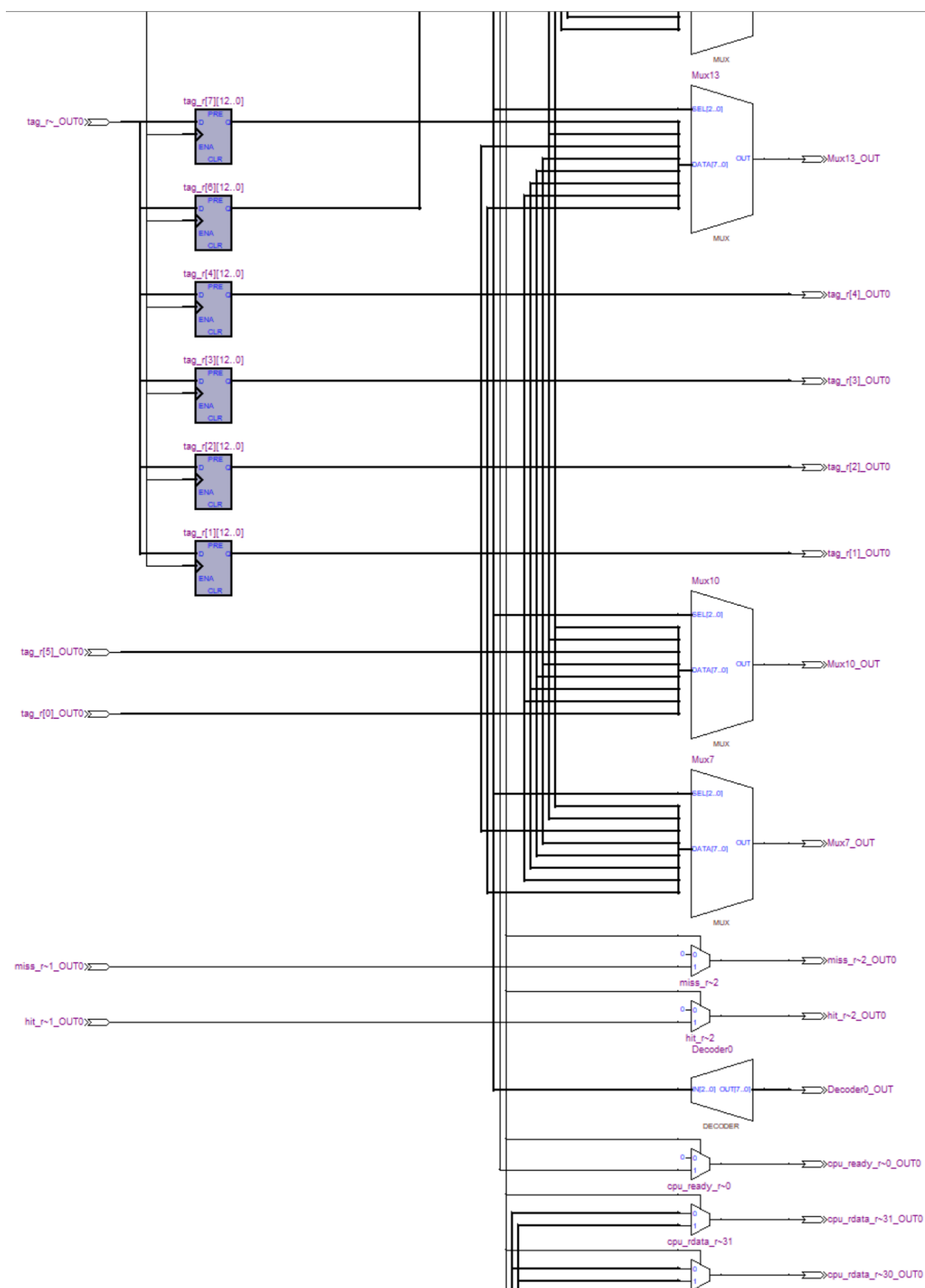


Рисунок 4.6 – Внутренняя организация строк и регистров кэш памяти, часть 2

4.2 Входные и выходные сигналы

В таблице приведены основные сигналы верхнего уровня. Имена выбраны развернутыми, чтобы по временной диаграмме было видно, какие линии относятся к процессорному интерфейсу, какие являются ответом кэша, а какие показывают обращение к основной памяти.

Таблица 4 – Назначение входных и выходных сигналов

Сигнал	Напр.	Разр.	Назначение
clock_i	in	1	Тактовый сигнал контроллера кэша и основной памяти.
reset_i	in	1	Сброс valid-битов, тегов, данных, выходных признаков и автомата управления.
cpu_cache_request_i	in	1	Запрос процессорного узла на чтение или запись.
cpu_write_enable_i	in	1	Тип операции: 0 – чтение, 1 – запись.
cpu_address_tag_set_i	in	16	Адрес процессора; A[15..3] – тег, A[2..0] – индекс строки.
cpu_write_data_to_cache_i	in	16	Слово, записываемое процессором.
cpu_read_data_from_cache_o	out	16	Слово, возвращаемое процессору при чтении.
cpu_cache_response_ready_o	out	1	Признак готовности результата операции.
cache_hit_signal_o	out	1	Импульс попадания выбранной строки кэша.
cache_miss_signal_o	out	1	Импульс промаха и необходимости

			обращения к памяти.
memory_request_from_cache_debug_o	out	1	Диагностический запрос контроллера к основной памяти.
memory_write_enable_debug_o	out	1	Диагностический признак записи в основную память.
memory_address_debug_o	out	16	Адрес, переданный из кэша в основную память.
memory_data_debug_o	out	16	Данные на интерфейсе основной памяти.

Процессорный запрос рассматривается как активный, когда `cpu_cache_request_i` равен единице. В этот момент адрес и данные записи должны быть устойчивыми относительно фронта тактового сигнала. Если `cpu_write_enable_i` равен нулю, операция трактуется как чтение; если равен единице, выполняется запись. При чтении вход `cpu_write_data_to_cache_i` не влияет на результат, но оставляется на диаграмме для контроля полного интерфейса.

Выход `cpu_cache_response_ready_o` формирует короткий признак завершения операции. Для попадания чтения он появляется сразу после проверки выбранной строки. Для промаха он появляется только после того, как слово получено из основной памяти и записано в строку кэша. Благодаря этому процессорный узел может отличить быстрый ответ кэша от ответа, требующего дополнительного цикла памяти.

4.3 Алгоритм работы

Алгоритм работы можно разделить на четыре основных режима: ожидание запроса, чтение с попаданием, чтение с промахом и запись. В состоянии ожидания контроллер не обращается к памяти и удерживает выход `ready` в нуле. При появлении запроса адрес фиксируется, из него выделяются индекс и тег, после чего выполняется проверка выбранной строки.

Таблица 5 – Алгоритм обработки обращения к кэш-памяти

№	Действие	Результат
1	После сброса очищаются valid-биты и выходные признаки.	Кэш считается пустым, любое первое чтение дает промах.
2	При запросе чтения адрес делится на TAG и INDEX.	INDEX выбирает строку, TAG сравнивается с сохраненным тегом.
3	Если valid=1 и теги совпадают, фиксируется попадание.	Данные выбранной строки выдаются процессору, ready=1, обращение к памяти не выполняется.
4	Если valid=0 или тег не совпал, фиксируется промах.	Формируется запрос чтения основной памяти по исходному адресу.
5	После получения данных из памяти выбранная строка обновляется.	В строку записываются DATA, TAG и valid=1; данные выдаются процессору.
6	При записи слово помещается в строку, выбранную индексом.	Кэш обновляется локально, тег и valid также получают новые значения.
7	Одновременно при записи формируется запрос к основной памяти.	Реализуется сквозная запись, поэтому память содержит актуальное слово.
8	Если другой адрес имеет тот же индекс, но другой тег, строка замещается.	Повторное обращение к старому адресу снова дает промах.

Управляющая часть работает как синхронный автомат. В режиме ожидания она сбрасывает служебный запрос к основной памяти и ожидает входной запрос процессора. Если обнаружено попадание при чтении, автомат не меняет состояние и сразу формирует готовность. Если обнаружен промах чтения, адрес сохраняется во внутреннем регистре, чтобы во время ожидания

памяти входной интерфейс мог уже измениться без искажения текущей операции.

При записи контроллер действует иначе: данные сразу помещаются в выбранную строку кэша, а затем передаются в основную память. Такой порядок удобен для демонстрации, потому что последующее чтение того же адреса показывает обновленное слово без дополнительной задержки, а повторное чтение после вытеснения доказывает, что слово было записано не только в кэш, но и в память.

Признаки hit и miss рассматриваются как диагностические импульсы текущей операции. Они не должны постоянно удерживаться в единице после завершения обращения, иначе по временной диаграмме было бы трудно определить, к какой операции относится конкретный признак. Поэтому в начале каждого такта контроллер сбрасывает эти признаки и заново устанавливает их только при активном запросе или завершении цикла промаха.

4.4 Реализация функциональных блоков

Блок разбора адреса выделяет индекс и тег. Для адреса 0010h младшие три разряда равны 000, поэтому используется строка 0. Адрес 0018h также имеет индекс 000, но другой тег. Именно поэтому обращение к 0018h вытесняет ранее загруженную строку для адреса 0010h.

Блок сравнения тега работает только с выбранной строкой. Если строка невалидна, попадание запрещено независимо от значения тега. Если строка валидна и тег совпадает, контроллер формирует hit и возвращает сохраненное слово. Если тег не совпадает, формируется miss и запускается цикл обращения к основной памяти.

Блок основной памяти используется как источник данных при промахах и как приемник данных при сквозной записи. Начальное содержимое памяти задано детерминированно: адрес 0010h возвращает слово 1110h, адрес 0018h – слово 1198h. После записи ABCDh по адресу 0010h это же значение должно возвращаться из памяти даже после вытеснения строки из кэша.

Строка кэша обновляется атомарно с точки зрения внешнего наблюдателя: после завершения промаха выбранная строка уже содержит новый тег, valid-бит установлен, а на выходе данных находится слово, соответствующее запрошенному адресу. Это важно для повторного обращения: если следующий запрос имеет тот же адрес, он уже не должен обращаться к основной памяти.

При конфликте индекса новая строка не ищет свободное место в других строках. Именно это отличает прямое отображение от ассоциативных вариантов. Аппаратная схема получается проще, но программная последовательность обращений может чаще вызывать промахи. В рамках лабораторной работы такое поведение считается корректным и должно быть явно видно на временной диаграмме.

Таблица 6 – Состав функциональных блоков устройства

Блок	Назначение
Разбор адреса	Выделяет TAG=A[15..3] и INDEX=A[2..0].
Массив valid-битов	Показывает, содержит ли выбранная строка корректные данные.
Массив тегов	Хранит старшую часть адреса для проверки попадания.
Массив данных	Хранит восемь 16-разрядных слов кэша.
Сравнение тега	Определяет hit или miss для выбранной строки.
Контроллер обращения	Формирует ready, hit, miss и управляющие сигналы основной памяти.
Основная память	Возвращает слово при промахе чтения и принимает слово при записи.

4.5 Функциональная проверка

Проверка построена как последовательность обращений процессорного узла к кэшу. Сначала выполняется первое чтение адреса 0010h. Так как после сброса valid-биты очищены, операция должна завершиться промахом и загрузкой слова 1110h из основной памяти. Повторное чтение того же адреса уже должно завершиться попаданием.

Далее выполняется запись W[0010h]=ABCDh. При сквозной записи это слово одновременно обновляет строку кэша и ячейку основной памяти. Затем выполняется чтение адреса 0018h, который имеет тот же индекс, что и 0010h, но другой тег. Это обращение вытесняет строку 0010h. Финальное чтение 0010h снова дает промах, однако возвращает уже ABCDh, потому что основная память была обновлена во время записи.

Контрольные адреса выбраны так, чтобы в одной диаграмме показать и положительный, и отрицательный результат сравнения тега. Адрес 0010h

сначала загружает строку, затем подтверждает попадание. Адрес 0018h отличается старшей частью, но имеет тот же индекс, поэтому он проверяет конфликт прямого отображения. Такая последовательность исключает случай, когда схема кажется исправной только из-за обращения к разным строкам.

На временной диаграмме необходимо контролировать не только `cru_read_data_from_cache_o`, но и `memory_request_from_cache_debug_o`. При попадании чтения запрос к памяти должен отсутствовать. При первом чтении и при обращении к 0018h запрос памяти появляется, так как данные отсутствуют в выбранной строке. При записи активируется `memory_write_enable_debug_o`, что подтверждает сквозную запись.

Таблица 7 – Контрольные операции функциональной проверки

Операция	Условие	Ожидаемый результат
Первое чтение	READ 0010h	miss=1, обращение к памяти, data=1110h.
Повторное чтение	READ 0010h	hit=1, data=1110h, запрос к памяти отсутствует.
Сквозная запись	WRITE 0010h, data=ABCDh	Строка кэша и основная память получают ABCDh.
Проверка записи	READ 0010h	hit=1, data=ABCDh.
Конфликт индекса	READ 0018h	miss=1, data=1198h, строка с индексом 0 замещается.
Повтор старого адреса	READ 0010h	miss=1, data=ABCDh загружается из основной памяти.

Таблица 8 – Интервалы временной диаграммы

Интервал	Операция	Ключевые признаки
0–20 нс	Сброс	valid=0, ready=0, hit=0, miss=0.
30–70 нс	READ 0010h	Первый промах, запрос памяти, возврат 1110h.
80–90 нс	READ 0010h	Попадание в уже заполненную

		строку.
110–140 нс	WRITE 0010h=ABCDh	Сквозная запись, обновление кэша и памяти.
150–160 нс	READ 0010h	Проверка обновленного слова ABCDh.
180–220 нс	READ 0018h	Промех из-за другого тега, замещение строки.
230–270 нс	READ 0010h	Повторный промах после замещения, возврат ABCDh из памяти.

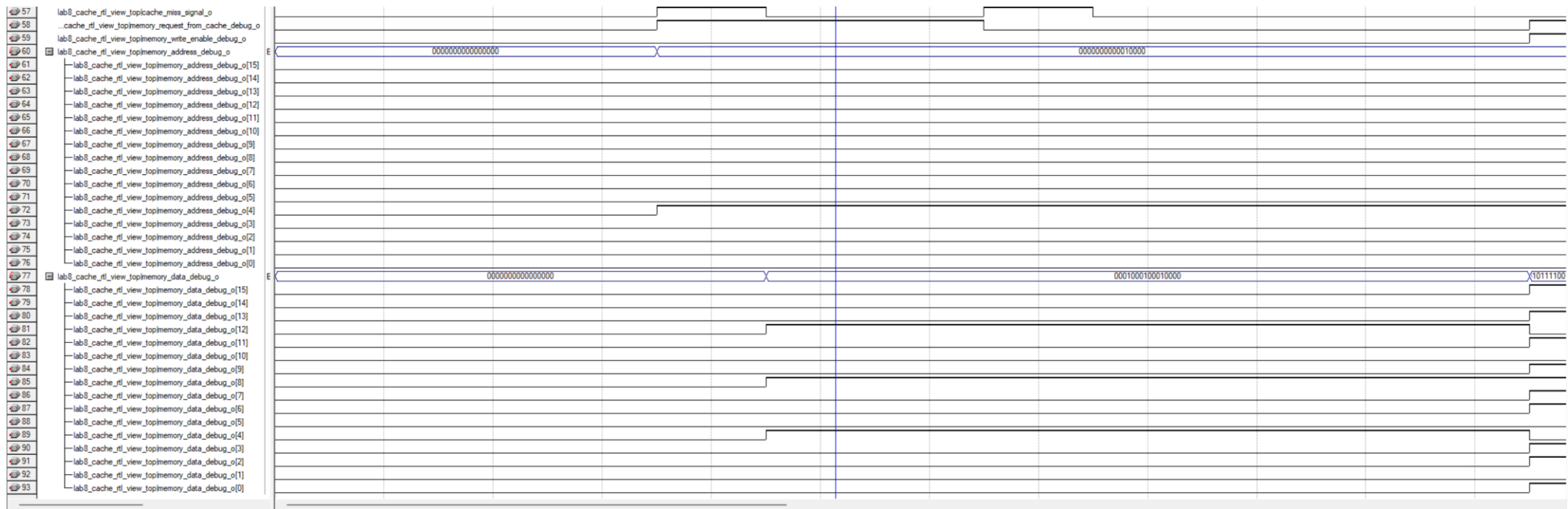


Рисунок 4.8 – Временная диаграмма выдачи данных после заполнения строки

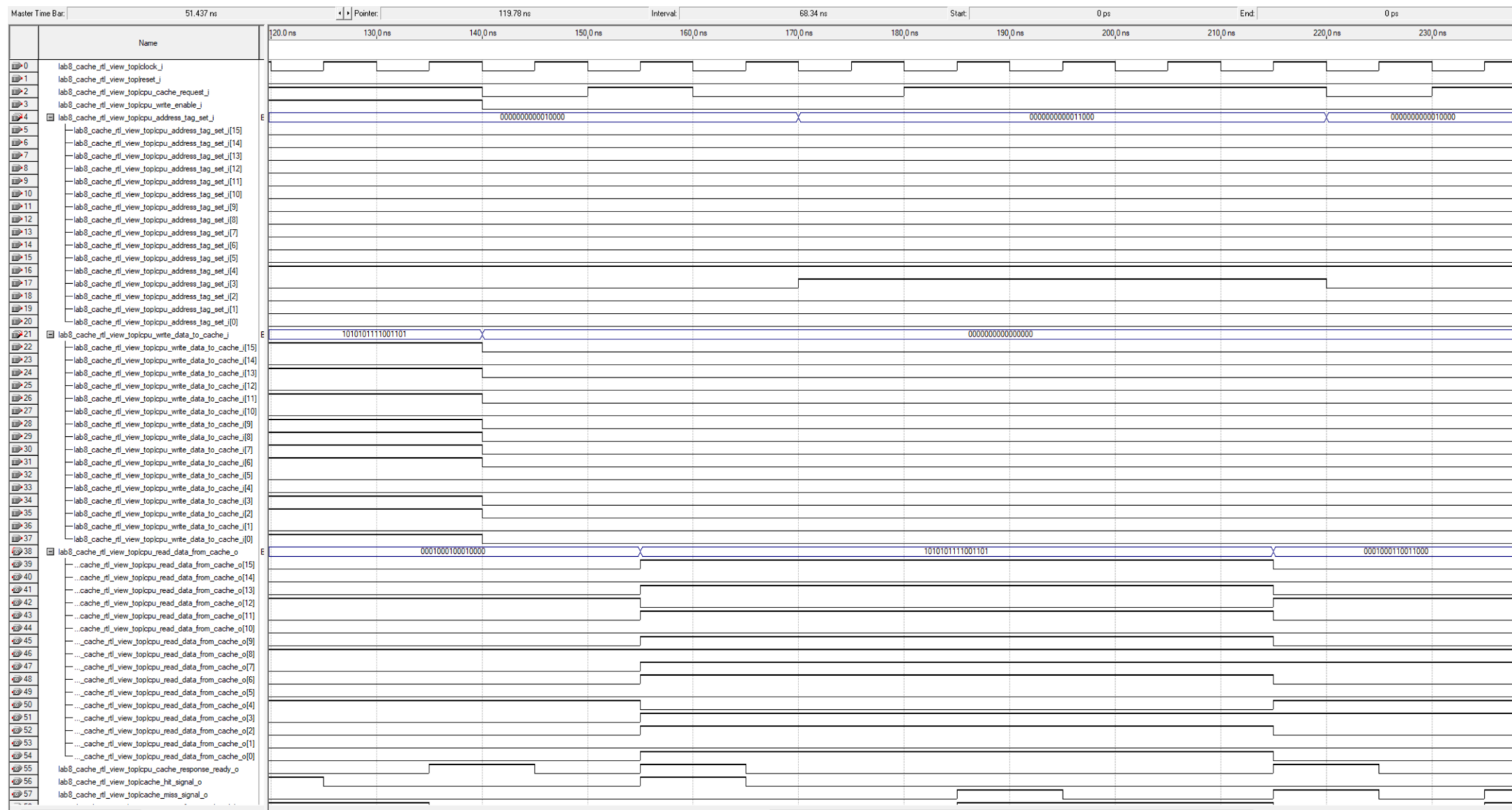


Рисунок 4.9 – Временная диаграмма повторного чтения и операции записи



Рисунок 4.10 – Временная диаграмма сквозной записи в основную память

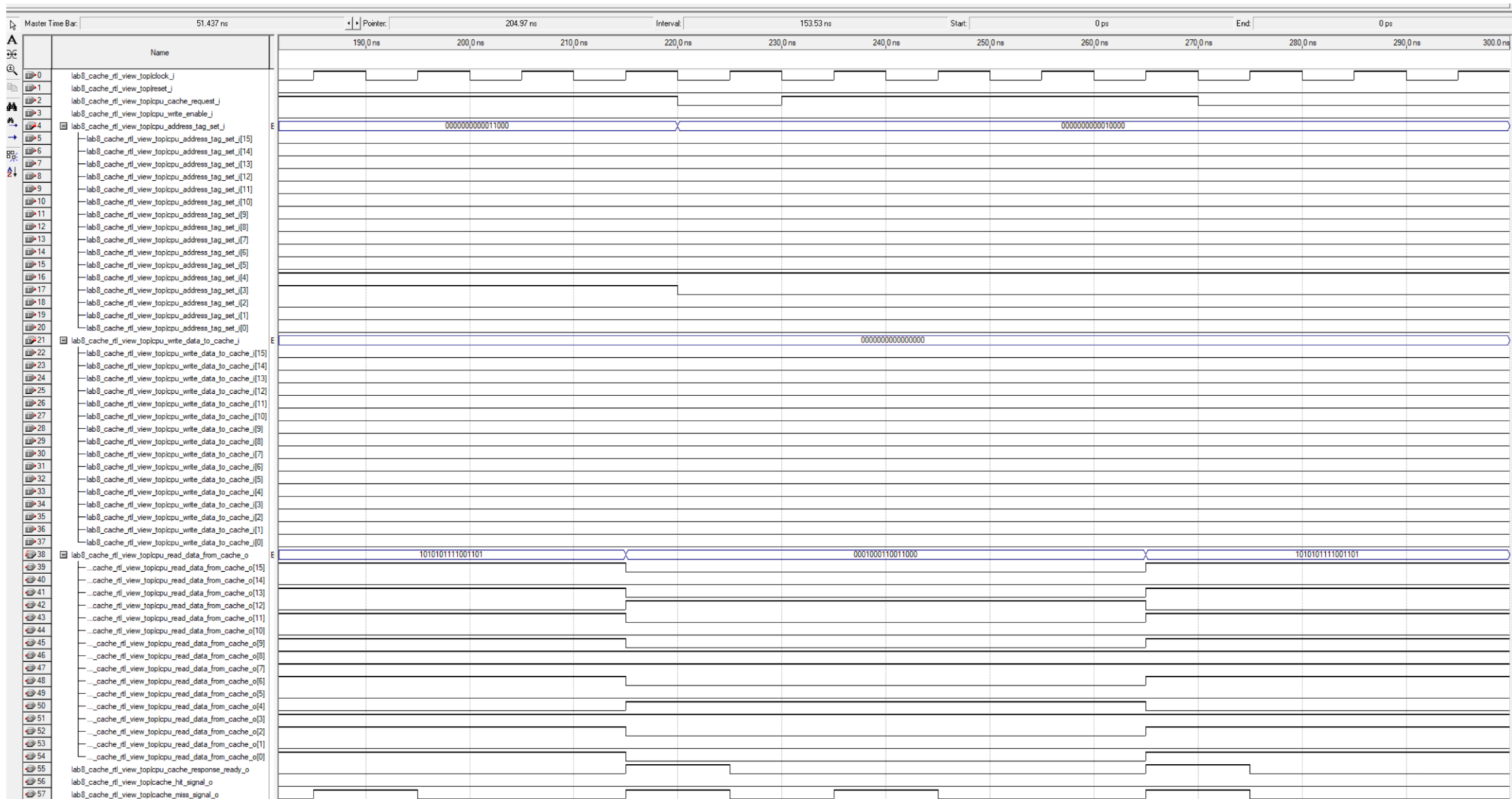


Рисунок 4.11 – Временная диаграмма конфликта индекса и замещения строки



Рисунок 4.12 – Временная диаграмма повторного промаха и возврата обновленного слова

На временных диаграммах видна требуемая последовательность событий. Для первого чтения адреса 0010h активируется признак промаха и запрос к памяти. После возврата слова 1110h строка получает valid-бит и тег, поэтому повторное чтение завершается попаданием. При записи ABCDh активируется признак записи в память, а последующее чтение того же адреса подтверждает обновление строки.

Адрес 0018h выбран специально: его младшие три бита совпадают с адресом 0010h. Поэтому оба адреса попадают в одну строку прямого отображения. При чтении 0018h старый тег заменяется новым, а при последующем чтении 0010h снова возникает промах. Возврат ABCDh в конце проверки подтверждает, что сквозная запись ранее обновила основную память.

4.6 Результаты синтеза

После сборки проекта выполнена полная компиляция. Ошибки компиляции отсутствуют. Предупреждения о незафиксированных физических выводах допустимы для лабораторной работы, так как проверяется функциональная схема устройства, а не разводка конкретной платы.

Основная часть ресурсов расходуется на регистры строк кэша, теги, valid-биты, регистры интерфейса и синхронную основную память. Использование блочной памяти связано с моделью основной памяти, хранящей 256 16-разрядных слов. Логическая часть контроллера занимает небольшую долю ресурсов выбранного семейства.

Таблица 9 – Результаты синтеза устройства

Показатель	Значение
Combinational ALUTs	200 / 12 480 (2 %)
Dedicated logic registers	374 / 12 480 (3 %)
Total pins	89 / 343 (26 %)
Total block memory bits	4 096 / 419 328 (<1 %)
Simulation time	300 ns
Vector input source	lab8_cache_wave_prefilled.vwf
Результат компиляции	Full compilation: 0 errors
Результат моделирования	0 errors

5 Вывод

В ходе лабораторной работы разработана кэш-память с прямым отображением для варианта 4. Устройство содержит восемь строк, каждая строка хранит valid-бит, 13-разрядный тег и 16-разрядное слово данных. Адрес разделен на поле TAG=A[15..3] и поле INDEX=A[2..0], что соответствует выбранному числу строк.

Разработанный алгоритм обеспечивает чтение при попадании, обработку промаха с загрузкой слова из основной памяти, сквозную запись и замещение строки при конфликте индекса. Контрольная последовательность обращений READ 0010h, READ 0010h, WRITE 0010h=ABCDh, READ 0010h, READ 0018h и READ 0010h показывает все основные режимы работы кэша.

Функциональная проверка подтвердила корректность признаков hit, miss и ready, а также правильность обмена с основной памятью. После записи ABCDh по адресу 0010h повторный промах после замещения возвращает именно ABCDh, что подтверждает работу сквозной записи. Полная компиляция проекта завершена без ошибок, поэтому разработанное устройство соответствует индивидуальному варианту и может использоваться как подсистема памяти учебной микро-ЭВМ.